Programmation 3D
Rendering

Objectif

L'objectif de ce cours est de décrypter les différentes étapes du processus de rendu d'une image 3D.

Techniques d'affichages

Techniques

Il existe deux principales techniques d'affichage relative à l'affichage d'un rendu 3D :

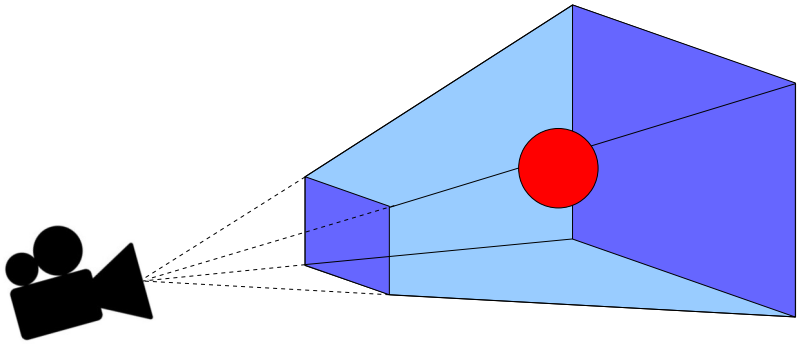
- La rastérisation
- Le ray-tracing

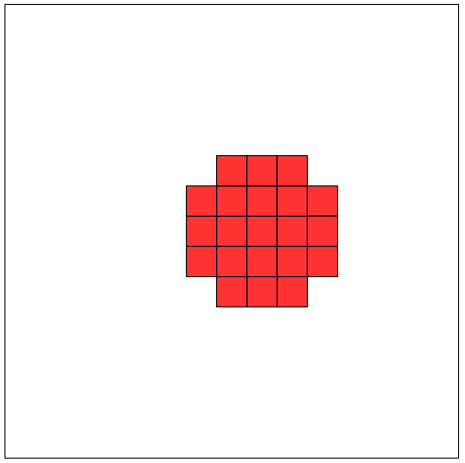
(Voir également : Le ray-marching)

Rastérisation

La rastérisation consiste à déterminer la surface occupée par un objet 3D sur la surface de rendu.

Les objets sont projetés dans l'espace de rendu.





Rastérisation

La rastérisation permet de :

- Rapidement tracer des surfaces.
- Eliminer les éléments hors caméra ou cachés.

En revanche, elle a les inconvénients suivants :

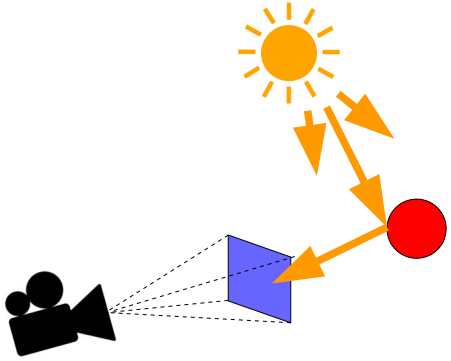
- Problèmes d'ordre de rendu sur la transparence.
- Nombre de triangles et de pixels rendus en millions.
- Calculs de lumières et d'ombres couteux et peu réaliste (pas de rebonds lumineux).

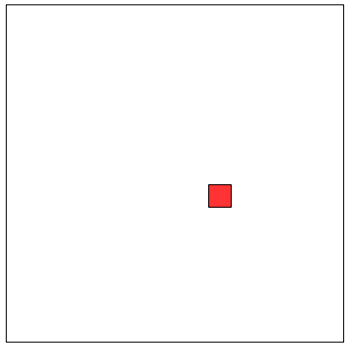
Rastérisation

Ray-tracing

Forward ray-tracing : Sens naturel. Les rayons lumineux partent d'une source pour arriver à l'oeil après un ou plusieurs rebonds.

La plupart des rayons n'arrivent pas jusqu'à l'oeil, et le chemin d'un rayon peut être complexe. Le forward ray-tracing est donc trop coûteux pour une application numérique.

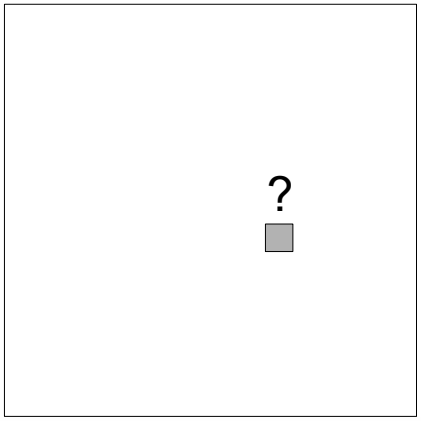


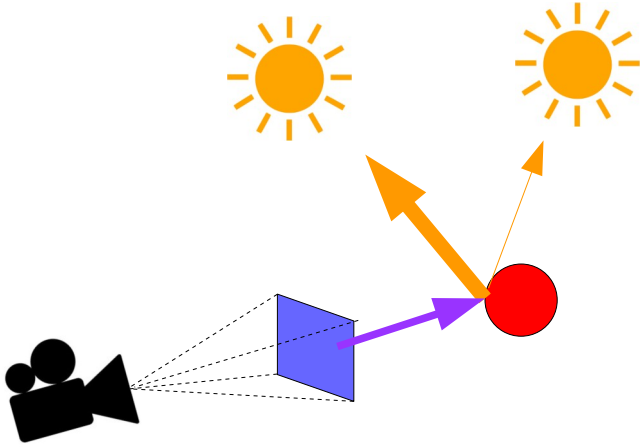


Ray-tracing

Backward ray-tracing : Les rayons lumineux partent d'un pixel et rebondissent un certain nombre de fois. La luminosité est évaluée à chaque point de contact.

Cette technique offre un bon résultat et est utilisable en temps réel.

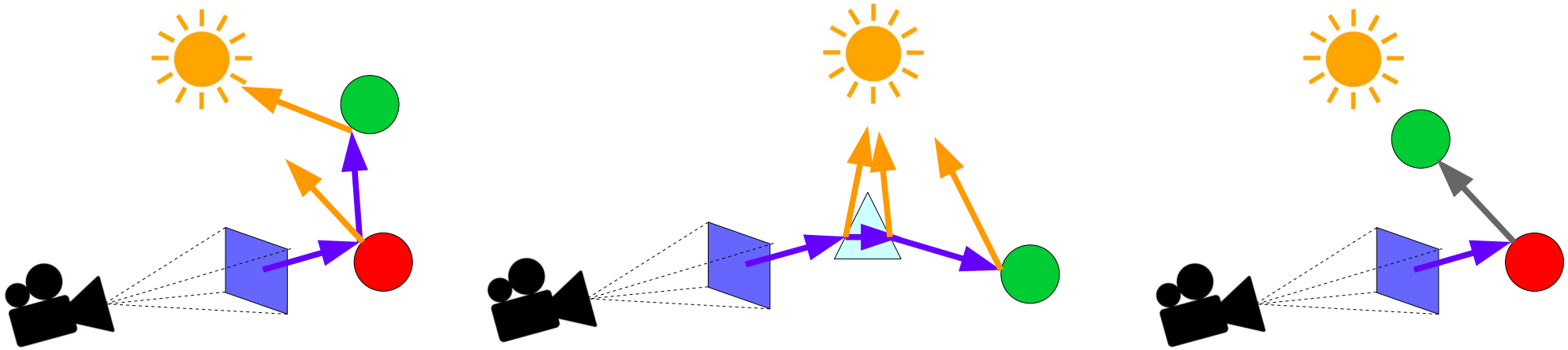




Ray-tracing

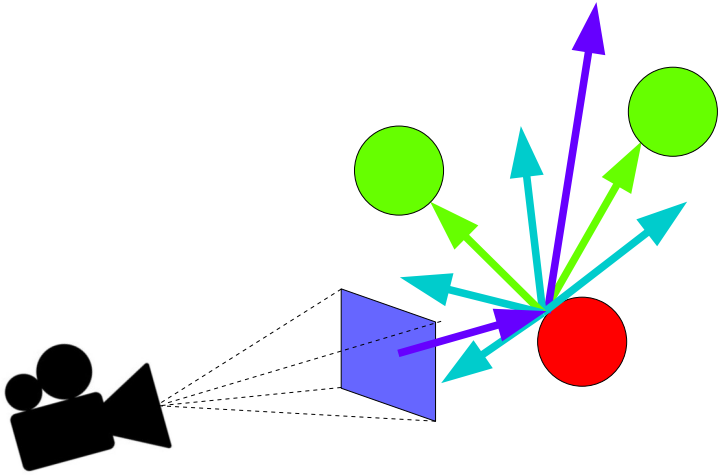
Un rayon peut :

- Rebondir sur plusieurs objets (reflets)
- Passer à travers les objets transparents (réfraction).
- Etre bloqué par un autre objet (ombres)



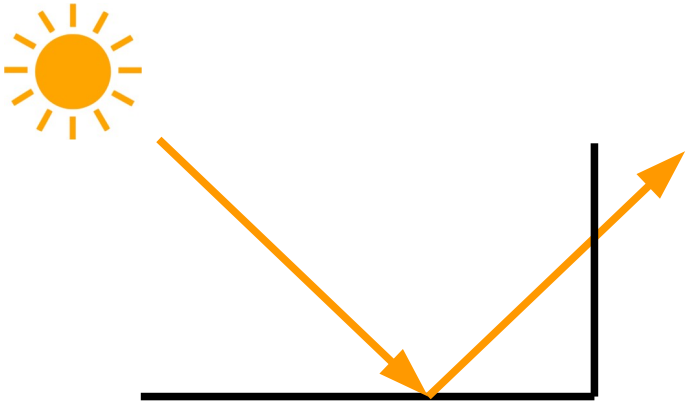
Ray-tracing

Pour affiner les calculs de lumières indirectes, plusieurs rebonds secondaires peuvent être tirés à partir d'un même point de contact.



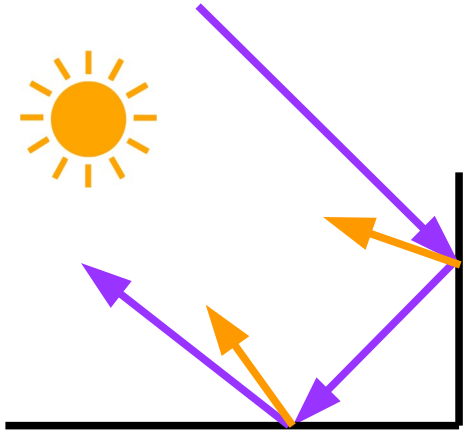
Ray-tracing

Rastérisation



Un seul rebond.

Ray-tracing



Rebonds multiples.

Ray-tracing

En résumé, le ray-tracing permet d'obtenir un résultat **photoréaliste** bluffant :

- Effets de transparence poussés (translucidité, réfraction, ...).
- Ombres naturelles.
- Reflets parfaits.

En revanche, son **coût** est encore **élevé** pour le temps réel :

- Beaucoup de rayons et rebonds calculés.
- Un coût dépendant d'avantage du nombre de pixels à afficher que du nombre d'objets sur la scène. (4K = 8M de pixels)

Ray-tracing

Global illumination

La **global illumination**, ou le meilleur des deux mondes :

Utilise la rastérisation pour rendre les objets rapidement.

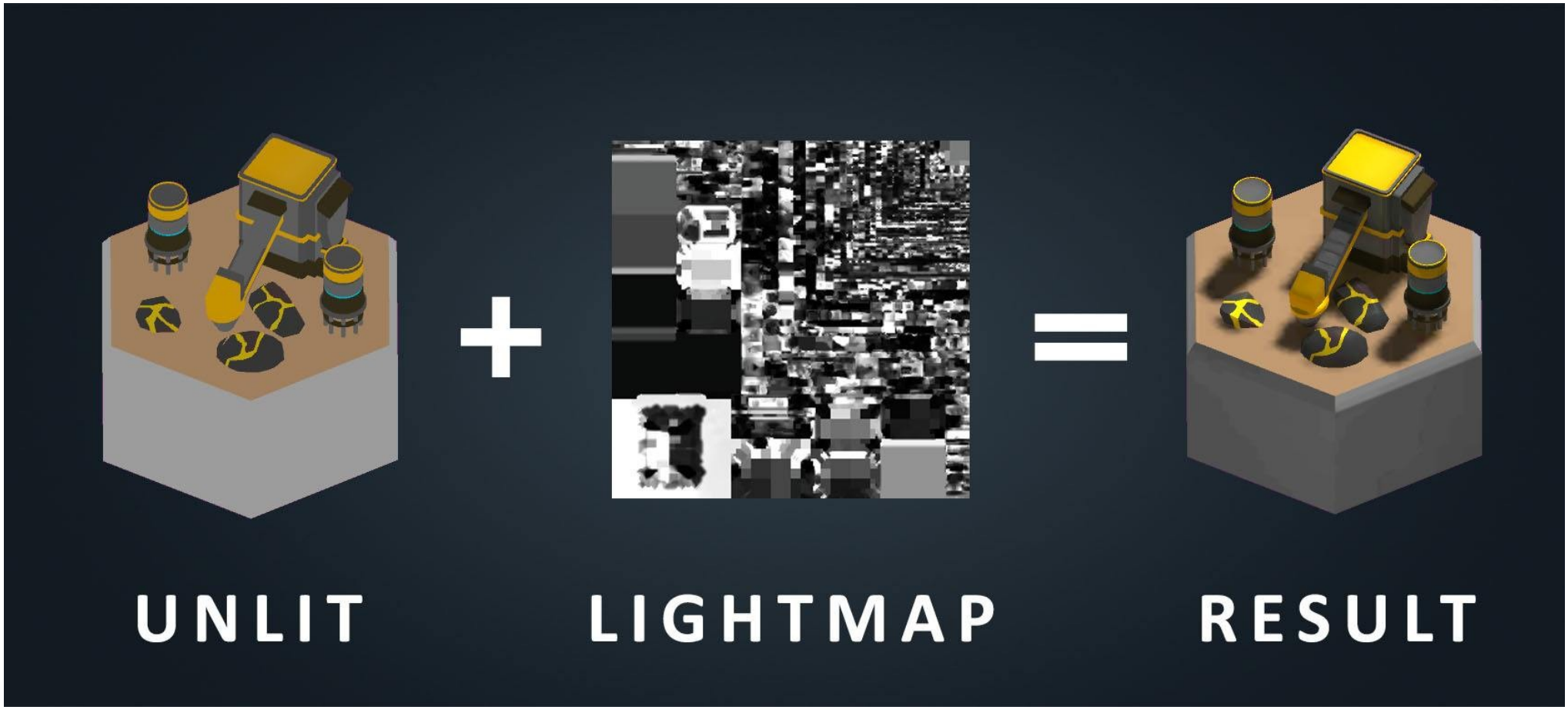
Précalcule les ombres et les éclairages indirects en s'appuyant sur le ray-tracing (path-tracing) pour affiner le rendu des objets statiques.

Global illumination

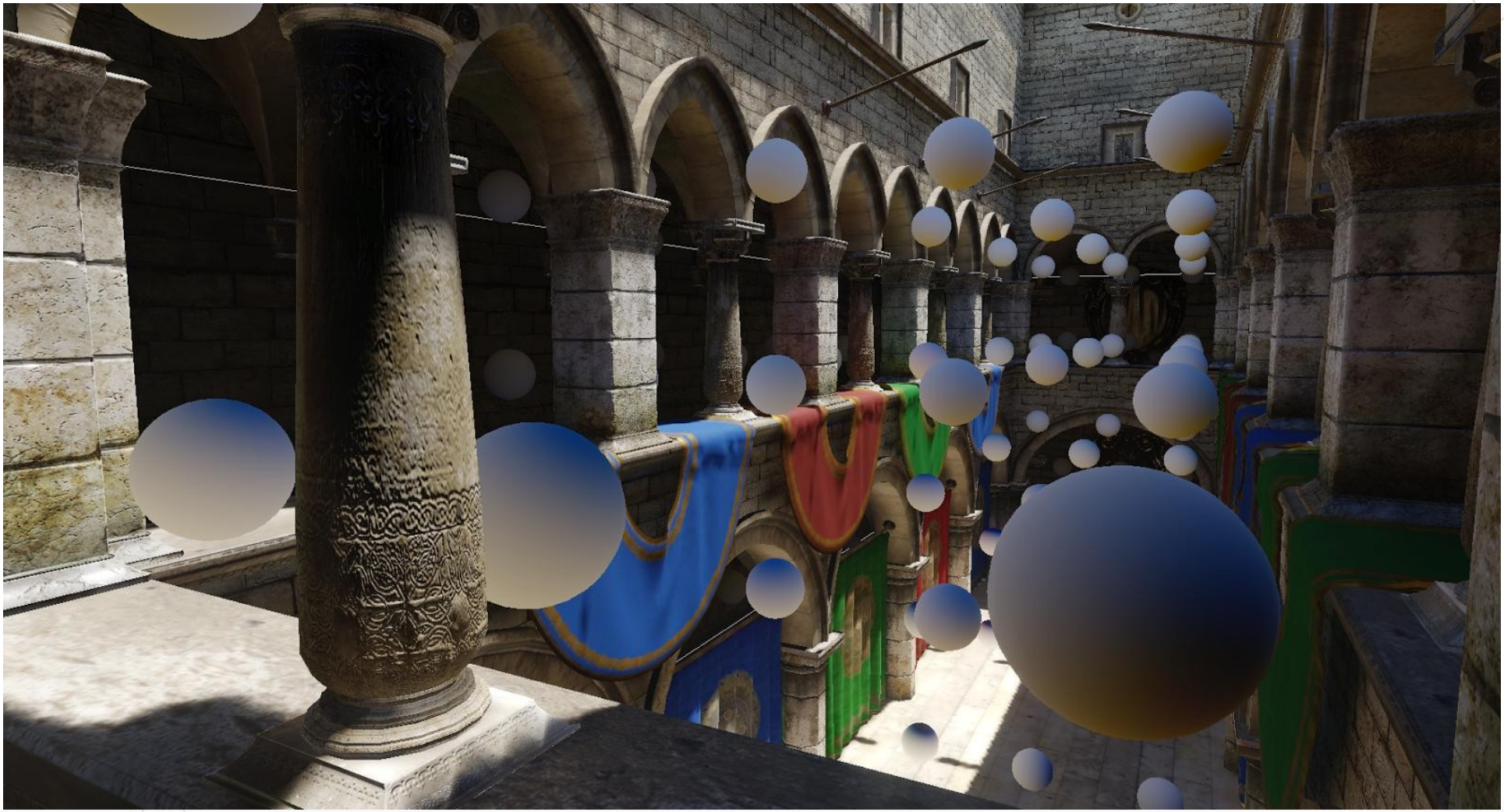
Deux concepts clés de la GI :

- **Lightmap** : Texture pré-rendue contenant les effets des sources lumineuses sur les objets statiques
- **Light probes** : Points de l'espace pour lesquels l'effet des sources lumineuses est pré-rendu pour éclairer les objets non-statiques.

Lightmap



Light Probes

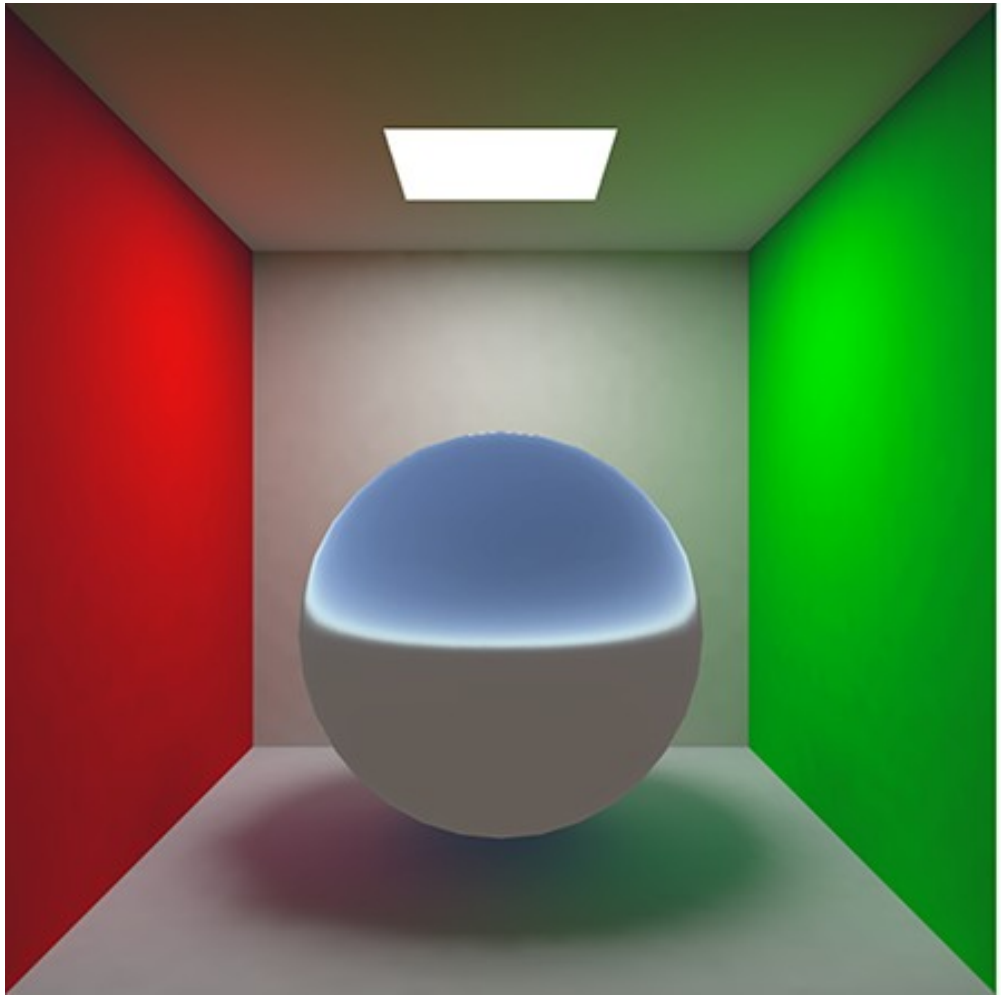


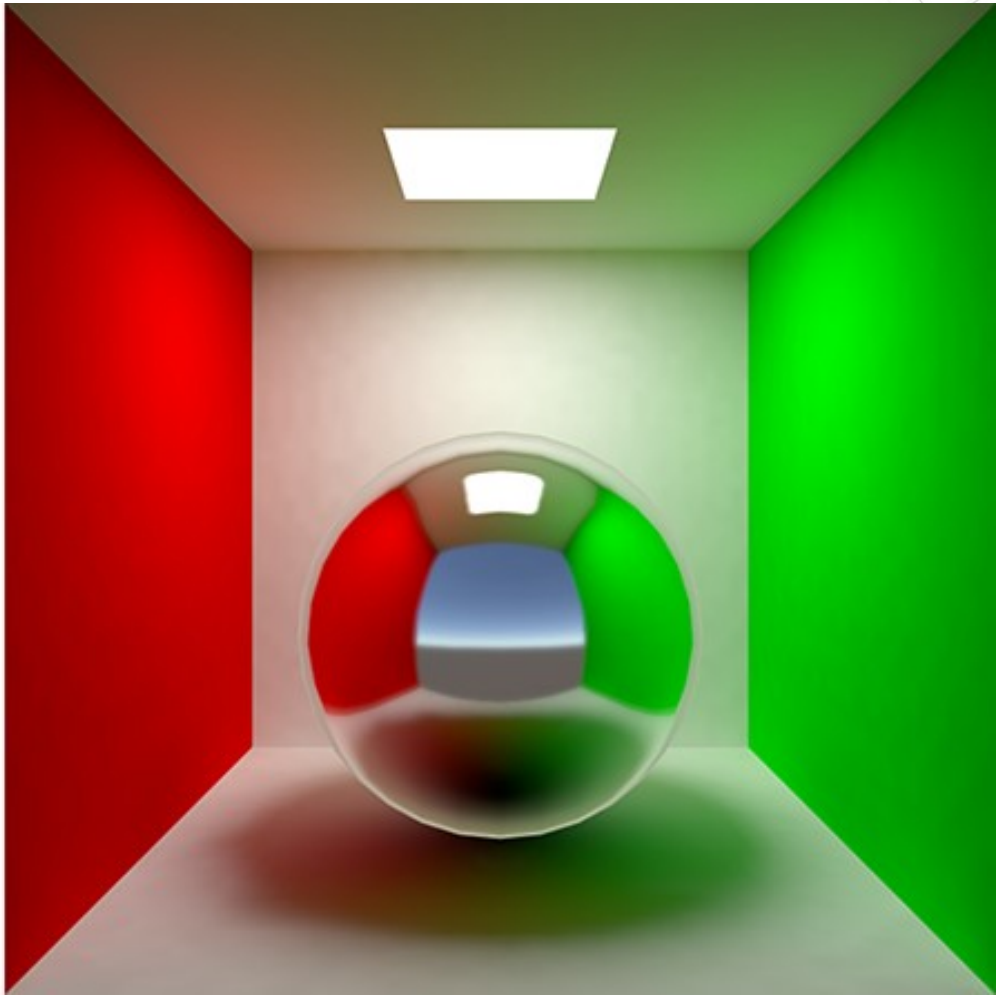
Réflexions

Pour assurer des réflexions nettes avec la rasterisation :

- Reflexion probes : Textures pré-rendues à 360° (cubemap) des objets statiques de la scène servant à détailler la réflexion de l'environnement des objets réfléchissants.
- Screen Space Reflections : Effet post-rendu s'appuyant sur les normales et la profondeur pour calculer des réflexions.

Reflection probe



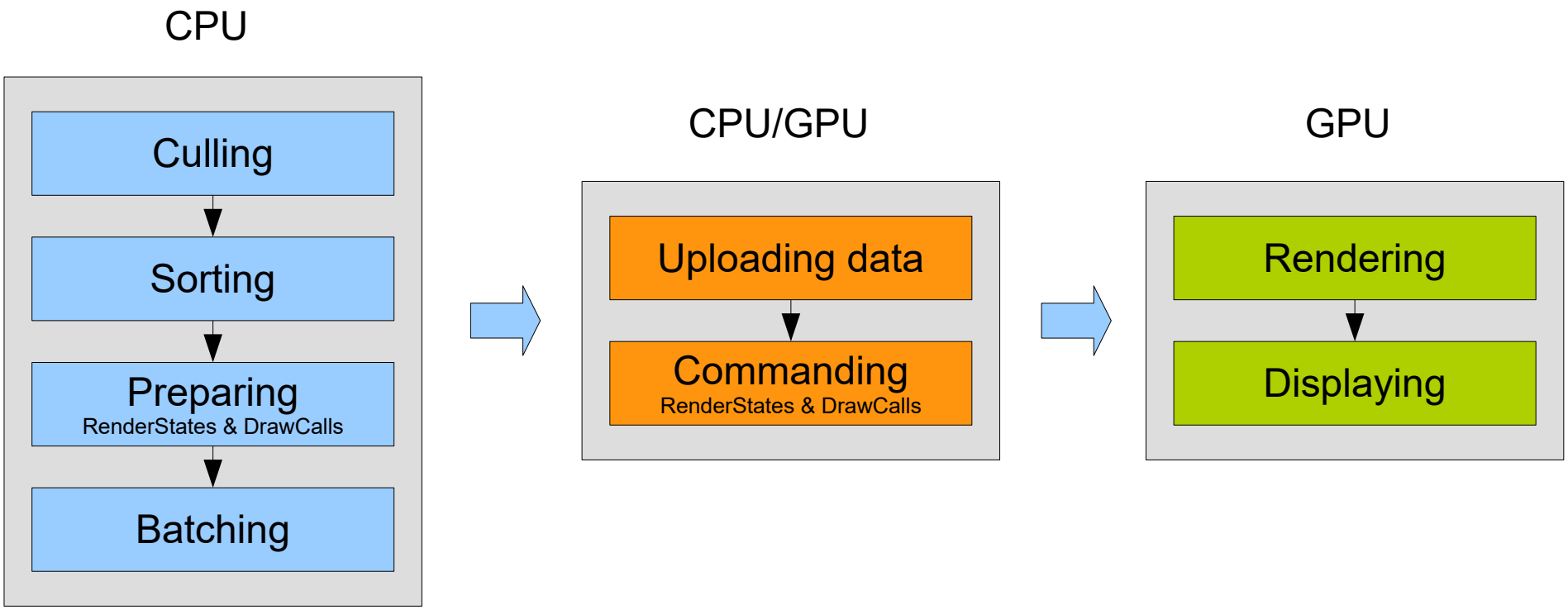


Screen Space Reflections

Etapes de rendu

Etapes de rendu

Etapes (très) simplifiées d'un pipeline de rendu :



Préparation CPU

Les étapes de préparation **CPU** :

- **Culling** : Détermine pour chaque objet 3D s'il doit être rendu (bounding box dans le frustum).
- **Sorting** : Défini un ordre d'affichage optimisé et cohérent.
- **Préparation des Draw Calls** : Pour chaque objet 3D à rendre, préparation d'une commande d'affichage pour le GPU.

Préparation CPU

Etapes de préparation **CPU** (suite) :

- **Batching** : Rassemble les Draw Calls partageant des caractéristiques similaires en lots, ou **batches.** Un batch regroupe plusieurs objets 3D qui seront rendu comme un objet 3D unique.

Upload CPU-GPU

Etapes de synchronisation **CPU-GPU** :

- **Upload** : Pour chaque objet à tracer qui n'est pas chargé sur carte graphique, transfert des infos en VRAM : mesh, textures, shaders, informations d'animation, ...

Commandes CPU-GPU

Etapes de commande **CPU-GPU, pour chaque batch** :

- **Préparation au rendu :** Le CPU indique au GPU quel shader et quelle configuration utiliser pour ce batch. C'est le **Render State**, comparable à un pinceau servant à tracer tout le batch (forme, couleur, caractéristiques).
- **Draw Call :** Les Draw Calls sont exécutés dans l'ordre d'émission par le CPU sur le GPU.

Etapes GPU

Etapes d'affichage **GPU** :

- **Rendering** : Le GPU exécute les commandes du CPU dans l'ordre de réception. Pour chaque ordre de Draw Call, le GPU exécute le code du shader paramétré et trace sur le frame buffer.
- **Affichage :** Une fois le rendu de la frame fini, le frame buffer est présenté à l'écran.

Ordre de rendu

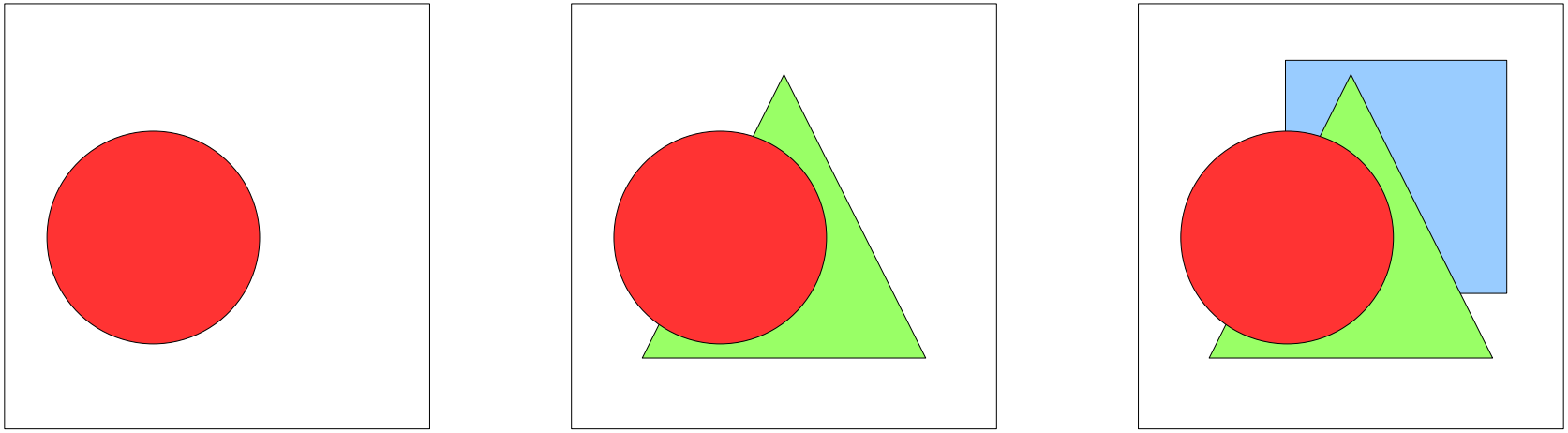
Ordre d'affichage

L'ordre d'affichage, ou **Render Queue Order**, dépend avant tout du paramétrage des objets 3D. On rendra dans l'ordre les objets :

- En arrière-plan
- Opaques
- En cut-out (alpha = 0 ou 1)
- Transparents (alpha)
- En overlay (alpha)

Ordre : Opaque

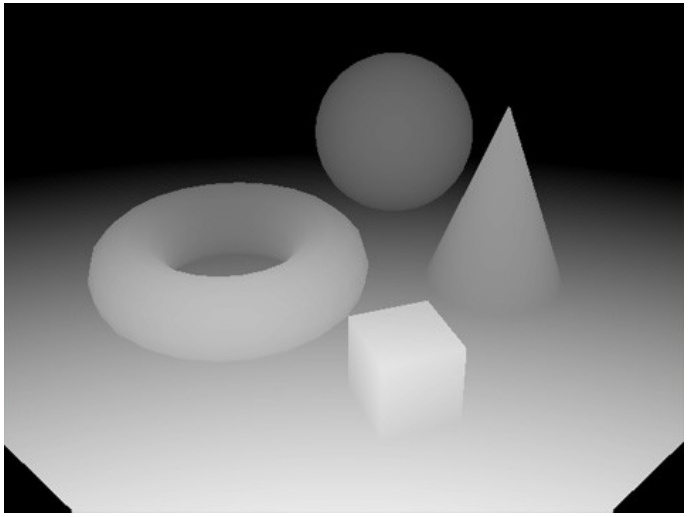
Les objets opaques sont "généralement" rendu du plus près au plus loin pour éviter de réécrire des pixels.



Z-Buffer

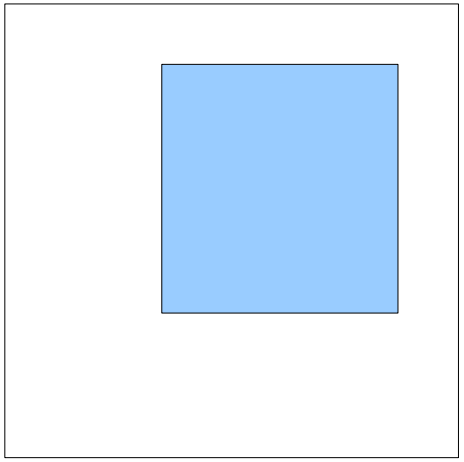
Chaque pixel du frame buffer est associé a une donnée de profondeur. C'est le **Z-Buffer**.

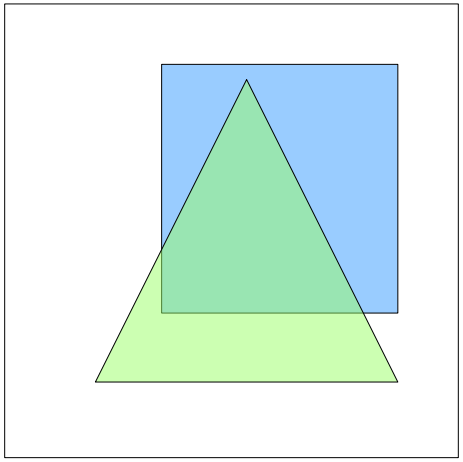
Lors du rendu, si la profondeur du pixel à rendre est supérieure à la valeur actuelle du Z-Buffer, il n'est pas rendu.

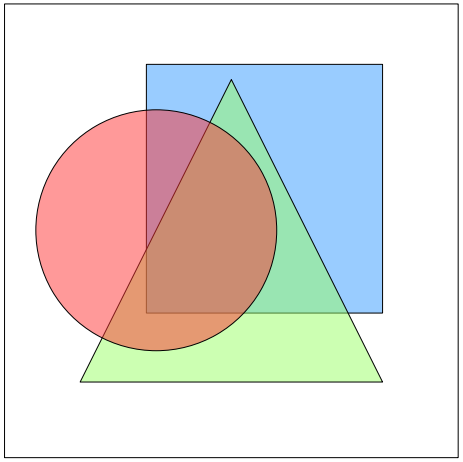


Ordre : Transparent

Les objets transparents sont rendus du plus loin au plus prêt pour pouvoir se mélanger avec les pixels déjà tracés.

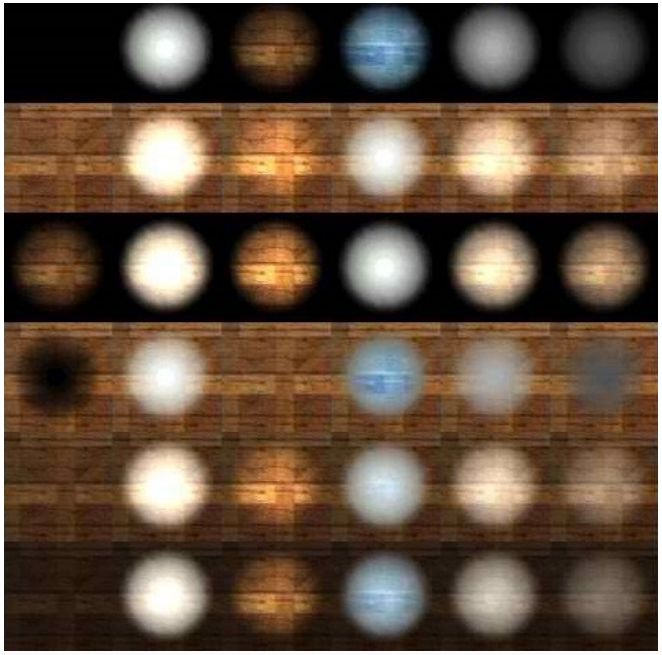






Blend

Lorsqu'un objet transparent est tracé, il combine sa couleur et celle de ce qui a déjà été tracé avec une opération de **Blend.**



Batching

Batching

Il est possible de regrouper certains objets 3D rendus par batch si les objets :

- Partagent le même material avec **la même configuration.**
- Ne sont pas retournés (scales négatifs)
- Font partis d'une même catégorie : Static, Dynamic, Particle system, trail renderer, ...

Dynamic batching

Au runtime, le CPU peut regrouper les meshs partageant une même configuration en un mesh unique pour les rendre en un seul draw call :

- Peu de vertices (< 300)
- Peu de data sur les vertices (< 5)

Couteux pour le CPU, léger pour le GPU.

Static batching

Les objets statiques (qui ne bougent pas) peuvent être regroupés à l'avance en mémoire CPU.

Les meshs batchés sont réunis en un jeu de données combinées transmis au GPU d'un bloc.

Réduit le nombre de Render State transmis au GPU (mais pas le nombre de draw calls).

Léger pour le CPU et GPU, couteux en RAM. Attention aux meshs répétés (i.e. forêt d'arbres).

Mesh combination

Il est possible de regrouper des meshs ensembles à l'avance en utilisant des outils de combinaison de meshs.

Implique un seul draw call, mais n'autorise pas le culling, contrairement au static batching.

Léger pour le CPU et le GPU si les objets doivent être rendus ensembles.

Idem sur la mémoire que le static batching.

GPU Instancing

Le GPU Instancing permet de tracer plusieurs fois un même mesh avec un même material et des variations mineures (couleur, position, scale) (i.e. forêts)

Léger pour le GPU quand applicable.

Le GPU Instancing n'est pas efficace sur de petits mesh (< 300 vertices)

Unity SRP Batcher

Réduit le nombre de changements de Render State transmis au GPU en regroupant les mesh par shaders et en persistant les materials en mémoire GPU.

Léger pour le CPU et le GPU.

A privilégier si supporté.

Choix du Batching

Les différents types de batching doivent être sélectionnés en fonction du type de projet et du hardware visés.

Par exemple, le statique batching et le dynamic batching sont :

- Efficaces sur mobile pour soulager la charge GPU.
- Moins efficaces voir coûteux sur les PC modernes où le GPU n'est pas le goulot d'étranglement.

Le batching doit être étudié et profilé !

Forward et deferred rendering

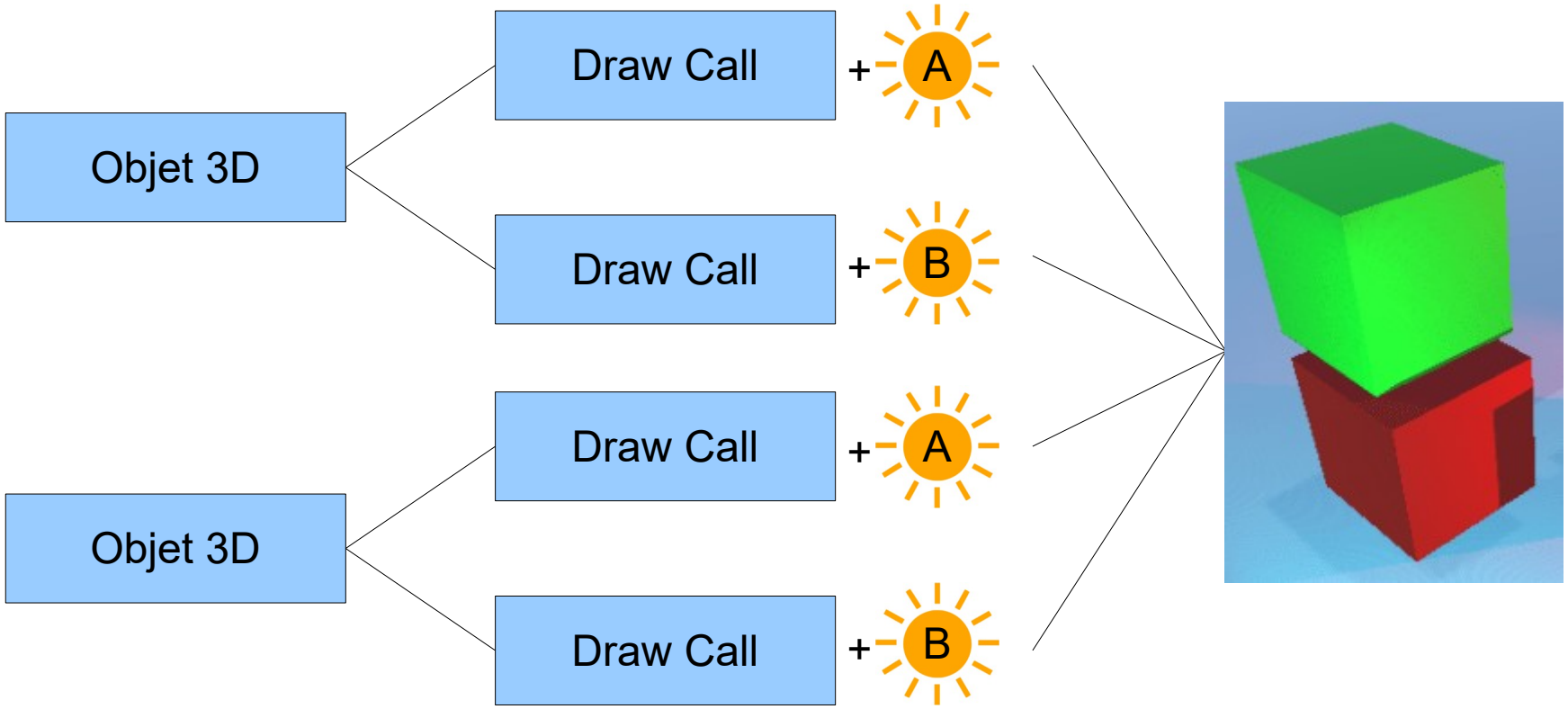
Forward rendering

Le forward rendering est un pipeline de rendu graphique éprouvé qui s'adapte à la majorité des situations de rendu.

Il trouve cependant un goulot d'étranglement important sur sa gestion des lumières :

- Les objets 3D sont rendus une fois par lumière pour chaque lumière à portée.

Forward rendering



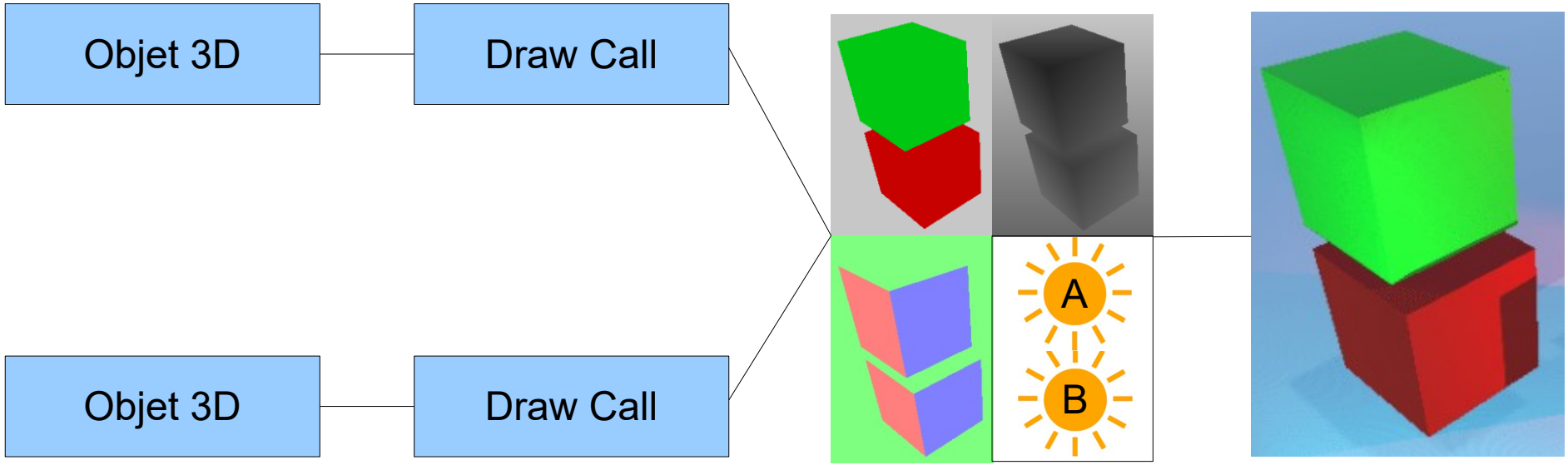
Deferred rendering

Le deferred rendering est un pipeline de rendu graphique plus récent couteux en performance mais pouvant gérer un grand nombre de lumières.

Les objets 3D sont rendus une seule fois dans différents buffers (normales, profondeur, couleur).

La lumière est ensuite appliquée à l'ensemble du rendu.

Deferred rendering



Deferred rendering

Le deferred rendering peut afficher plusieurs dizaines de lumières et matériaux simultanément.

Il a cependant quelques inconvénients importants :

- Il ne gère pas la transparence (mix forward-deferred),
- Il ne supporte pas les algorithmes d'anti-aliasing classiques,
- Sans lumières, il est plus couteux que le forward rendering.
- Il rend une qualité inférieure pour différents effets : Ambient occlusion, surfaces emmissives, ombres, normales, ...

VRAM

VRAM

La VRAM (Video RAM) est la mémoire de la carte graphique.

Les meshs, materials, shaders et textures utilisés pour le rendu sont stocké en VRAM pour un accès plus rapide au GPU.

Dès qu'un nouvel objet 3D est affiché, ses données sont chargées en VRAM.

VRAM : Swap

Dès qu'un nouvel objet 3D est affiché, ses données sont chargées en VRAM.

Si une ressource n'est plus utilisée, elle est déchargée de la VRAM.

Si la VRAM est pleine, les données des objets 3D sont échangés fréquemment entre la VRAM et la RAM : C'est le **Swap**. Le taux de rafraichissement chute fortement.

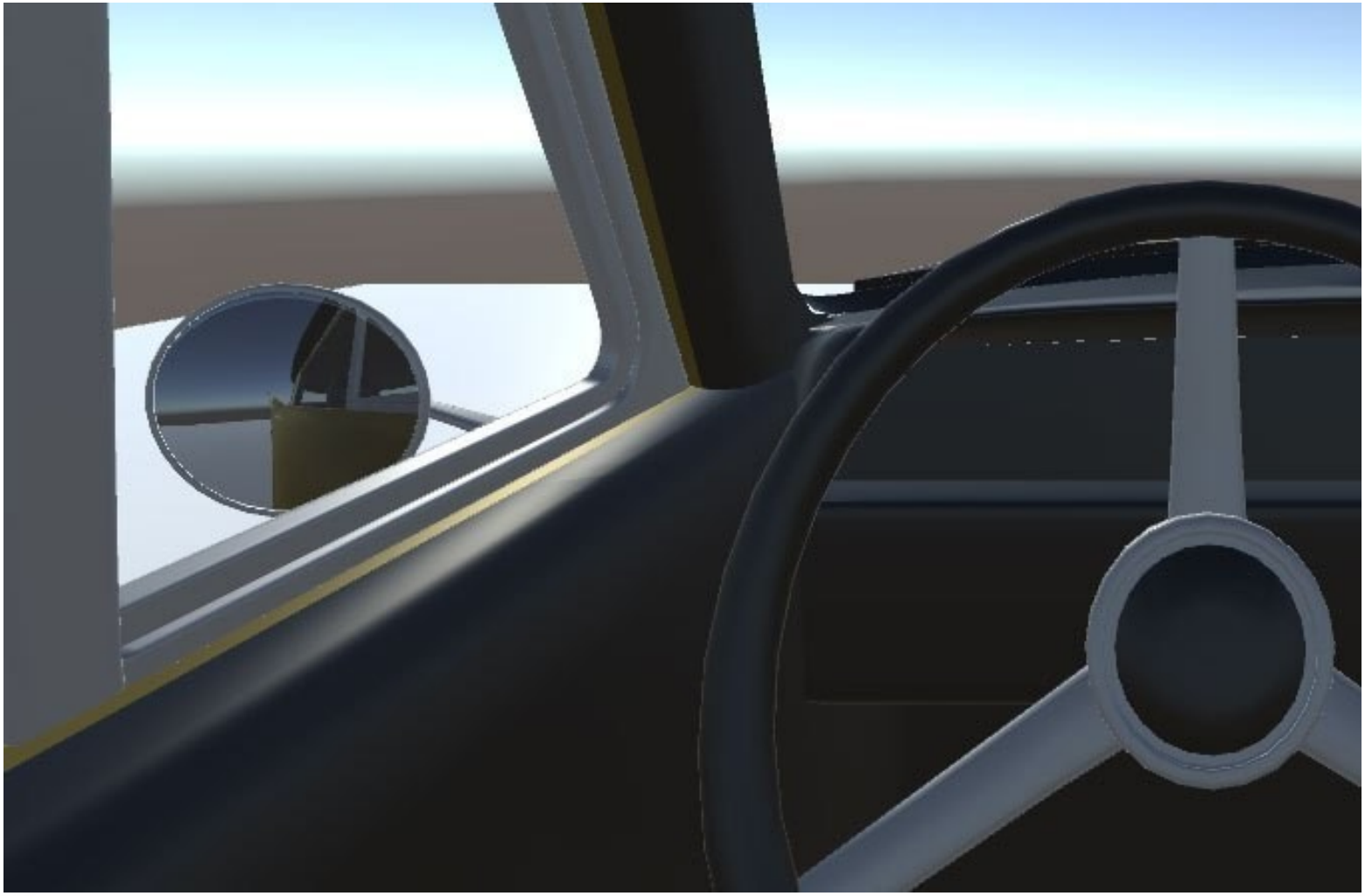
Render Texture

Frame Buffer Object

Il est possible de demander à une caméra de tracer dans une texture de rendu, ou **frame buffer object**, plutôt que de tracer à l'écran.

Sous Unity, ces textures de rendu sont appelées simplement **RenderTexture.**

Frame Buffer Object



Exercices

Frame Debugger

Dans un de vos projets Unity en cours ou terminer, utiliser le Frame Debugger pour observer le rendu :

- Lancer le jeu en mode "Scène"
- Atteindre une frame potentiellement couteuse en 3D
- Dans "Windows->Analysis", sélectionner le Frame Debugger
- Appuyer sur Enable
- Utiliser la barre de slide pour observer les Draw Calls.

Project Zero:
Fatal FrameBuffer

Dans un nouveau projet Unity :

- Ajouter un terrain, un CharacterController, une sphère "fantôme" et un quad qui servira d' "écran de sécurité".
- Ajouter une deuxième caméra au-dessus du terrain. Modifier les layers rendus par les caméras pour que seule la deuxième caméra puisse rendre la sphère "fantôme".
- Créer une RenderTexture et l'assigner comme surface de rendu cible de la deuxième caméra.
- Afficher le résultat de la RenderTexture sur le quad "écran de sécurité".

La sphère n'apparait que sur l'écran de sécurité ^(;..;)^